The Journal of Open Source Software

Source Known Identifiers: A Three-Tier Identity System for Distributed Applications

Duran Serkan Kılıç [1]

1 Independent Researcher

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Source Known Identifiers (SKIDs) are 64/128-bit unique identifiers designed for distributed applications that require the **ideal identifier properties**. Standard UUID schemes carry limited or no origin context. SKIDs instead embed contextual metadata (entity type, application Id, application instance Id, timestamp, and sequence number) directly into the identifier. This design supports runtime diagnostics and origin tracing without external lookups.

The DRN.Framework (Kılıç, 2024) provides a reference implementation in C#/.NET, organized as a three-tier identity model:

1. **Source Known Identifier (SKID)**: a 64-bit (`long`) sortable identifier embedding entity type, timestamp, instance identity, and sequence counter.
2. **Source Known Entity ID (SKEID)**: a SKID augmented with an entity type discriminator and a BLAKE3 keyed MAC for tamper-evident integrity verification at trust boundaries.
3. **Secure SKEID**: a SKEID encrypted with AES-256 (single-block PRP) for confidential external exposure.

Each tier aligns with a trust boundary: database (SKID), trusted internal environment (SKEID), and external consumers (Secure SKEID). The implementation integrates with Domain-Driven Design (Evans, 2003) patterns through the `SourceKnownEntity` abstract base class, with `SourceKnownId` and `SourceKnownEntityId` structs providing parsed identity representation.

Statement of Need

Distributed applications need unique identifiers that satisfy the **ideal identifier properties**:

- storage-efficiency
- chronological sortability for indexing
- origin-metadata-embedding
- zero-lookup verifiability
- confidentiality to external consumers
- multi-century addressability

No current scheme combines all of these constraints except SKIDs.

SKIDs target platform engineers, software architects, distributed systems developers and researchers who need identifiers that support runtime diagnostics, origin tracing, and auditing with cryptographic integrity and confidentiality. The reference implementation is in C#/.NET, but the SKID design is language-agnostic and portable to any runtime.

DRAFT

State of the Field

Distributed identifier schemes have evolved since the original UUID specification. RFC 9562 (Davis et al., 2024) standardized UUID versions 1, 3–8, with v7 introducing timestamp-ordered UUIDs that address the B-tree fragmentation problems of random v4 UUIDs.

Twitter's Snowflake architecture (Twitter Engineering, 2010) pioneered timestamp-prefixed, worker-partitioned 64-bit IDs for high-throughput systems.

ULIDs (Feerasta, 2016) and CUIDs (Elliott, 2022) add timestamp-prefixed uniqueness in a string-friendly format.

None of these approaches simultaneously satisfies the **ideal identifier properties** required for a complete distributed identity system.

Software Design

Bit Layout

The identity system operates at two levels:

SKID (64-bit `long`, database tier):

Field	Bit(s)	Width	Purpose
Sign / Epoch half	63	1 bit	Epoch half indicator for smooth epoch transitions
Timestamp	62–32	31 bit	Second precision, epoch-relative (~136 years)
App Id	31–26	6 bit	Application identifier (max 63)
AppInstance Id	25–21	5 bit	Instance discriminator (max 31)
Sequence	20–0	21 bit	Per instance monotonic counter (2,097,152/s)

The SKID provides chronological sortability (timestamp-first ordering) and uniqueness (instance + sequence within an entity scope, e.g. a database table).

SKEID (128-bit `Guid`, trusted / external tiers):

Field	Byte(s)	Width	Purpose
SKID upper half	0–3	32 bit	SKID bits 63–32 (epoch half, timestamp)
Entity Type	4	8 bit	Domain entity classification
Epoch	5	8 bit	Epoch index (~34,842 years total span)
MAC[0]	6	8 bit	BLAKE3 keyed MAC byte 0
Version	7	8 bit	`0x8D` (UUID V8, RFC 9562 §5.8)
Variant	8	8 bit	`0x8D` (RFC 9562 §4.1 variant)
MAC[1–3]	9–11	24 bit	BLAKE3 keyed MAC bytes 1–3
SKID lower half	12–15	32 bit	SKID bits 31–0 (App Id, App Instance Id, Seq)

The SKEID layout adds an explicit entity type discriminator and a BLAKE3 keyed MAC for tamper detection.

Secure SKEID (128-bit `Guid`, external tier):

Field	Byte(s)	Width	Purpose
Ciphertext	0–15	128 bit	AES-256 encrypted SKEID (pseudorandom bytes)

DRAFT

The entire SKEID layout is encrypted as a single AES block; no internal structure is visible to external consumers.

Lifespan and Epoch Support

The 31-bit timestamp field provides ~68 years per half; the sign bit doubles this to ~136 years per epoch while preserving monotonic sort order in signed 64-bit representation. SKEID byte 5 carries an 8-bit epoch index, where each value selects a 136-year window from 2025-01-01, giving 256 epochs and roughly 34,842 years of total coverage. This two-level time addressing gives the identity system a multi-century lifespan without ID collisions or sorting degradation. Clock-drift protection handles backward time jumps at two levels. Minor drifts (under 3 seconds) freeze the timestamp until the wall clock catches up. Critical drifts force the application instance to shut down and restart with a new instance ID. This prevents duplicate or out-of-order identifiers.

Cryptographic Choices

- **BLAKE3 MAC** (O'Connor et al., 2020): Selected for performance (~3.5× faster than HMAC-SHA-256 in keyed hashing benchmarks on small inputs), keyed mode suitability, and 256-bit security margin. The MAC is truncated to 32 bits, providing a $1/2^{32}$ false-positive rate, sufficient for identifier-scope tamper detection where full collision resistance is not required.
- **AES-256-ECB (Single Block)** (National Institute of Standards and Technology, 2001): For Secure SKEIDs, the entire 128-bit identifier is encrypted as a single AES block. ECB mode is secure for single-block encryption, avoiding the complexity and nonce-management overhead of modes like GCM or CTR. AES-256 provides 128-bit post-quantum security (Apon et al., 2022).
- **Collision Guard**: When AES-256 encryption produces ciphertext that coincidentally contains the unsecure SKEID marker bytes (`0x8D` at positions 7-8), the system iterates the variant byte within the RFC 9562 variant range (`0x8E-0xBF`), re-encrypting until the collision is resolved. This deterministic resolution preserves the ability to distinguish secure from unsecure SKEIDs without ambiguity.

The reference implementation currently uses a single key pair (BLAKE3 MAC key + AES-256 key). Key-ring rotation with multiple active key versions and graceful rollover is planned as future work.

Three-Tier Trust Model

```
  Database                Trusted Internal          External / Public

┌──────────┐           ┌──────────────┐          ┌──────────────┐
│ SKID     │ Generate  │ SKEID        │ ToSecure │ Secure SKEID │
│ Sortable │──────────►│ BLAKE3 MAC   │──────────►│ + AES-256    │
│ long     │◄──────────│ Tamper-evident│◄─────────│ Encrypted    │
└──────────┘   Parse   └──────────────┘ ToUnsecure└──────────────┘
```

All four operations are defined in `ISourceKnownEntityIdOperations` and implemented by `SourceKnownEntityIdUtils`:

- **`Generate(long id, byte entityType)`**: packs SKID bits, entity type, epoch, and version/variant markers into a 128-bit `Guid`, then computes and embeds the BLAKE3 keyed MAC to produce a SKEID.
- **`Parse(Guid entityId)`**: auto-detects the tier (checks for plaintext version/variant markers first, falls back to AES-256 decryption), verifies the MAC, and reconstructs the `SourceKnownEntityId`.
- **`ToSecure(SourceKnownEntityId id)`**: encrypts the SKEID with AES-256-ECB (idempotent; returns unchanged if already secure).

DRAFT

- **`ToUnsecure(SourceKnownEntityId id)`**: decrypts a Secure SKEID back to its plaintext SKEID form (idempotent; returns unchanged if already unsecure).

These operations produce two structs defined in `DRN.Framework.SharedKernel`:

- **`SourceKnownId`**: the parsed SKID, exposing `Id` (`long`), `CreatedAt` (`DateTimeOffset`), `InstanceId` (`uint`), `AppId` (`byte`), and `AppInstanceId` (`byte`).
- **`SourceKnownEntityId`**: wraps a `SourceKnownId` (`Source`) with `EntityId` (`Guid`), `EntityType` (`byte`), `Valid` (`bool`), and `Secure` (`bool`), providing the full parsed identity across all three tiers.

DRN.Framework Integration

The reference implementation lives in `DRN.Framework.SharedKernel` (interfaces, base classes), `DRN.Framework.Utils` (cryptographic, timestamp, sequence operations and bit packing), and `DRN.Framework.EntityFramework` (persistence integration). Key integration points:

- `SourceKnownEntity`: abstract base class for all DDD entities and aggregates
- `ISourceKnownEntityIdOperations`: abstraction for `Generate`, `Parse`, `ToSecure`, and `ToUnsecure`
- `[EntityType(byte)]` attribute: compile-time entity type registration
- `SourceKnownRepository<TContext, TEntity>`: generic repository base providing CRUD operations, SKEID validation, transparent Guid-to-long ID conversion, and cursor-based pagination (`PaginateAsync`, `PaginateAllAsync` with `EntityCreatedFilter` support)
- `SourceKnownIdValueGenerator`: EF Core value generator that automatically assigns SKIDs to new entities during `SaveChanges`; no manual ID management is required
- `DrnSaveChangesInterceptor`: EF Core interceptor that auto-assigns SKID and SKEID on insert
- `DrnMaterializationInterceptor`: EF Core interceptor that reconstructs the full SKEID from the stored SKID and the entity's type on database read

Research Impact Statement

No existing identifier scheme simultaneously satisfies the **ideal identifier properties**. SKIDs provide these properties in a single protocol, replacing separate layered schemes or operations. An Internet-Draft (Kılıç, 2026b), prepared for independent submission, provides reproducible test vectors and pseudocode for cross-platform implementation. BenchmarkDotNet measurements on .NET 10 (Apple M2) show SKID generation at 24.5 ns (~12x faster than UUID v7 at 292.6 ns) and unsecure SKEID generation at 155.9 ns (~1.9x faster) despite embedding metadata and a BLAKE3 MAC, while Secure SKEID with AES-256 encryption completes in 439.1 ns. The 21-bit sequence field caps generation at 2,097,152 IDs per second per instance; full-throttle benchmarks confirm that the sequence manager applies backpressure when this limit is reached, preserving uniqueness under sustained load. The open-source implementation, CI/CD pipeline, NuGet packages, unit and integration tests provide materials for independent evaluation and reproducibility.

AI Usage Disclosure

The following AI tools and LLM models have been used in the development of this software and the preparation of this paper. SKIDs were added to the DRN-Project roadmap on 19 November 2023; the subsequent two-plus years of development effort were supported by the following models:

- **Software development, documentation, and testing**: Anthropic Claude Opus 4.6, Claude Opus 4.5, Claude Sonnet 4.6, Claude Sonnet 4.5, Google Gemini 3.1 Pro, Gemini 3.0 Pro,

DRAFT

Gemini 3.0 Flash, Qwen 3, DeepSeek R1, OpenAI o1 and OpenAI 4o were used across the DRN-Project development lifecycle, including the SKIDs implementation, as agentic coding assistants for code generation, refactoring, code review, test scaffolding, and documentation drafting. All generated code, tests, and documentation were reviewed, validated, and modified by the author.

- **Paper authoring**: Claude Opus 4.6 and Gemini 3.1 Pro were used to draft, structure, and refine sections of this paper. All technical claims were verified against the implementation, and final editorial decisions were made by the author.
- **AI governance**: DiSC OS (Kılıç, 2026a) (Distinguished Secure Cognitive OS), an author-designed behavioral framework, was used to guide all AI-assisted workflows. With DiSC OS, security-first principles, TRIZ-based conflict resolution, and structured decision-making across code generation, review, and documentation tasks are enforced.

All AI-assisted outputs were reviewed, validated, and refined by the author. The architectural design, cryptographic choices, three-tier trust model, and all technical decisions are solely the work of the author. The author takes full responsibility for the correctness and originality of the work.

Acknowledgements

This work received no external financial support. GitHub CodeQL and SonarCloud provided static analysis and code quality scanning that strengthened the security posture of the implementation. CodeRabbit provided AI code review that contributed to the overall code quality.

References

Apon, D., Moody, D., Perlner, A., Paquin, R., & Smith-Tone, D. (2022). *Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process* (NIST IR 8413-upd1). NIST. https://doi.org/10.6028/NIST.IR.8413-upd1

Davis, K., Peabody, B., & Leach, P. (2024). *Universally Unique IDentifiers (UUIDs)* (RFC 9562). IETF. https://www.rfc-editor.org/rfc/rfc9562

Elliott, E. (2022). *CUID2: Secure, collision-resistant ids*. https://github.com/paralleldrive/cuid2

Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley. ISBN: 0321125215

Feerasta, A. (2016). *ULID: Universally Unique Lexicographically Sortable Identifier*. https://github.com/ulid/spec

Kılıç, D. S. (2024). *DRN-Project: Distributed Reliable .Net*. https://github.com/duranserkan/DRN-Project

Kılıç, D. S. (2026a). *DiSCOS (Distinguished Secure Cognitive OS)*. https://github.com/duranserkan/DRN-Project/blob/master/.agent/rules/DiSCOS.md

Kılıç, D. S. (2026b). *Source Known Identifiers (SKIDs)*. https://github.com/duranserkan/DRN-Project/blob/master/docs/draft-skid-00.md

National Institute of Standards and Technology. (2001). *Advanced Encryption Standard (AES)* (FIPS 197). NIST. https://csrc.nist.gov/publications/detail/fips/197/final

O'Connor, J., Aumasson, J.-P., Neves, S., & Wilcox-O'Hearn, Z. (2020). *BLAKE3: One function, fast everywhere*. https://github.com/BLAKE3-team/BLAKE3-specs/blob/master/blake3.pdf

DRAFT

Twitter Engineering. (2010). *Announcing Snowflake*. https://blog.twitter.com/engineering/en_us/a/2010/announcing-snowflake

DRAFT